

Project Specification & Technical Report

Connected Car AI Support Console

1. Project Background & Motivation

In the modern automotive industry, connected vehicle platforms (e.g., Tesla App, MyHyundai, MyBMW, FordPass) integrate telematics, billing registries, mobile client applications, and vehicle-side Electronic Control Units (ECUs). When a remote command fails (e.g., doors fail to unlock, remote-start is blocked, pairing issues, or navigation is offline), support technicians must query multiple database layers:

1. Billing Registries (Subscriptions): Confirming if the customer's remote access subscription is active.
2. eSIM Registries (Telematics): Checking the cellular RSSI strength and whether the vehicle is online.
3. Customer Database (CRM): Evaluating customer profiles, linked vehicle IDs, and support ticket counters.
4. Knowledge Indexes (RAG): Searching through troubleshooting manuals for symptom correlations.

Manual diagnostics

This project implements the Connected Car AI Support Console, an automated diagnostics console powered by LangGraph and Google Gemini LLM. It orchestrates a multi-agent system that classifies complaints, selectively queries systems based on intent planner decisions, queries mock database registers, pulls RAG reference manuals, and synthesizes root causes with actionable resolutions.

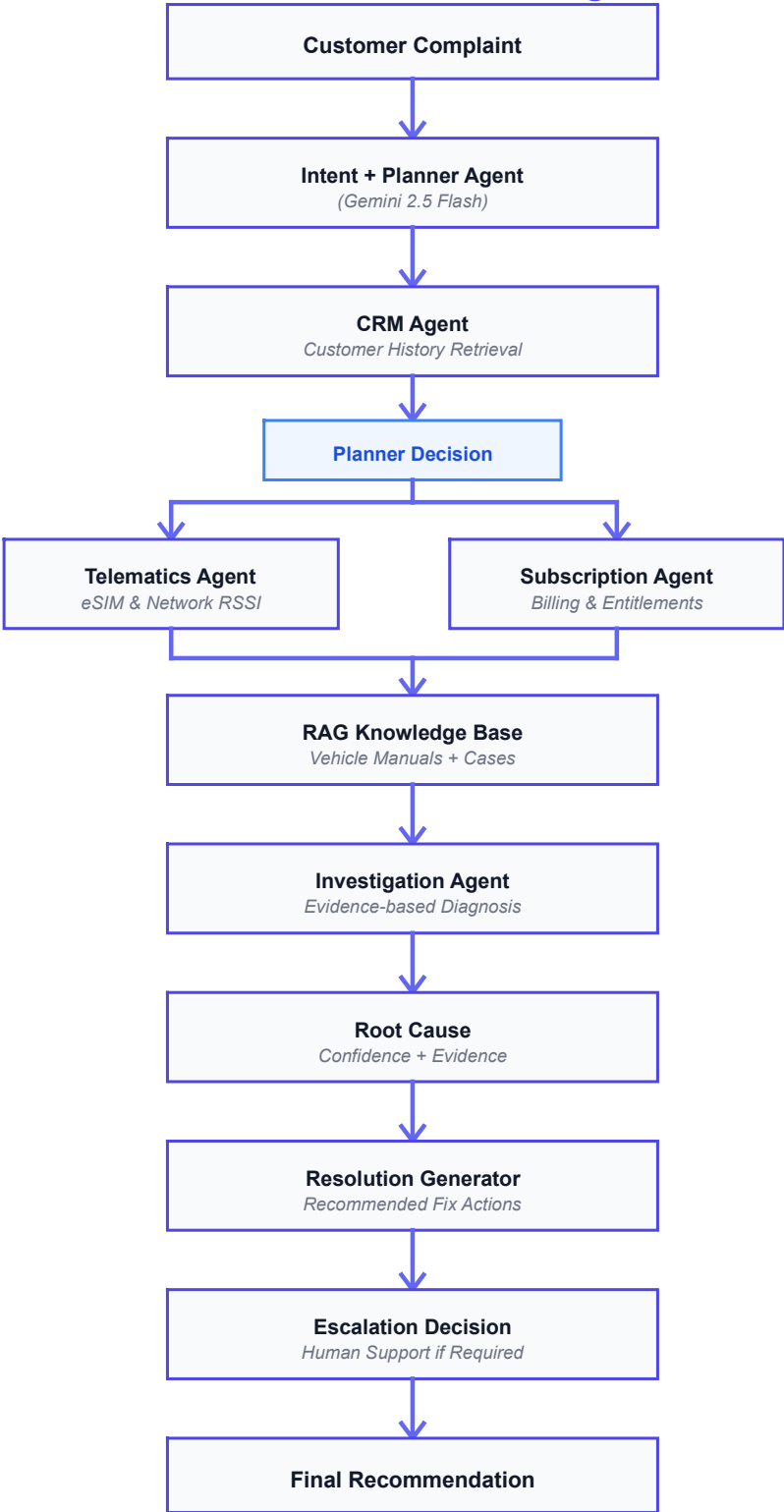
2. Workflow Architecture

The support console consists of a frontend dashboard (Streamlit) and a backend orchestration graph (LangGraph).

2.1 Connected Car AI Workflow Architecture

Below is the database-style block diagram showing the data registers, tables with keys (Primary Key [PK], Foreign Key [FK]), and processing modules.

2.1 Connected Car AI Workflow Architecture Diagram



2.2 Shared State Definition (`AgentState`)

The orchestrator maintains the context of the diagnostics run inside a shared state declared in graph/state.py:

```
from typing_extensions import TypedDict

class AgentState(TypedDict):
    customer_query: str          # Complaint text entered by the customer          customer_id: str
    # Customer identification code (e.g., C001)          vehicle_id: str          # Vehicle
    identification code (e.g., V001)          planner_decision: dict          # Decision flags generated
    by the Intent Agent:          # e.g. {"crm": True, "telematics": True,
    "subscription": True}
    issue_category: str          # Classified category of the problem
    severity: str                # Threat level (low, medium, high, critical)
    confidence: float            # Confidence of the intent classification step
    crm_data: dict               # Customer records returned by the CRM Agent
    telematics_data: dict        # Vehicle sensor frames returned by the Telematics Agent
    subscription_status: str      # Status returned by the Subscription Agent (active/expired)
    kb_context: str              # Diagnostic guidelines retrieved from the RAG Knowledge
    Base
    root_cause: str              # Final identified root cause of the
    breakdown          resolution: str          # Final action instructions
    root_cause_confidence: float # Synthesis confidence level
    evidence_used: list          # Checklist items proving the root cause
    investigation_steps: list     # Sequential execution log steps of the pipeline
```

3. Node-by-Node Orchestration Flow

LangGraph manages the orchestration via a Directed Acyclic Graph. Each node corresponds to a specific AI Agent or logic routine.

Node 1: Intent & Planner Node (`intent\_agent.py`)

- Objective: Classify client complaint and decide the optimal diagnostic pathway.
- Rule Engine:
 - app_pairing / remote_control_issue -> Trigger CRM, Telematics, Subscription checks.
 - subscription_issue -> Trigger CRM, Subscription checks.
 - door_lock_issue -> Trigger CRM only.
 - vehicle_start_issue / charging_issue / connectivity_issue -> Trigger CRM, Telematics checks.
- State Updates: Sets issue_category, severity, confidence, and planner_decision.

Node 2: CRM Profile Node (`crm\_agent.py`)

- Objective: Query customer CRM profile if planner_decision["crm"] is True.
- Action: Queries crm.json mock database.
- State Updates: Populates crm_data and appends trace records to investigation_steps.

Node 3: Telematics Node (`telematics\_agent.py`)

- Objective: Pull vehicle sensor logs if `planner_decision["telematics"]` is True.
- Action: Queries `telematics.json` mock database.
- State Updates: Populates `telematics_data` with parameters (battery percentage, online status, network rssi).

Node 4: Subscription Validator Node (`subscription\_agent.py`)

- Objective: Check billing status if `planner_decision["subscription"]` is True.
- Action: Queries `subscriptions.json` database.
- State Updates: Populates `subscription_status`.

Node 5: Knowledge Base RAG Node (`knowledge\_agent.py`)

- Objective: Retrieve relevant troubleshooting instructions matching the classified issue category.
- Action: Dynamically loads manual text files from the `data/kb/` directory (e.g. `door_lock_issue.txt`, `app_pairing.txt`).
- State Updates: Populates `kb_context`.

Node 6: Investigation & Synthesis Node (`investigation\_agent.py`)

- Objective: Analyze the gathered evidence and generate the root cause and repair resolution.
- Priority Logic:
 1. Real-world sensor inputs (e.g. subscription state, network logs).
 2. KB common symptoms.
 3. LLM synthesis from KB fallback.
- State Updates: Sets `root_cause`, `resolution`, `root_cause_confidence`, and `evidence_used`.

4. Databases & Data Layer Schema

4.1 CRM Register (`data/crm.json`)

Maps a customer's ID to their name, vehicle, and case history.

```
{
  "C001": {
    "name": "Hemanth N",
    "vehicle_id": "V001",
    "previous_tickets": 3
  }
}
```

4.2 Subscription Register (`data/subscriptions.json`)

Maps a customer's ID to their service entitlement status.

```
{
  "C001": {
    "status":
    "expired"
  },
  "C002": {
    "status":
    "active"
  }
}
```

```
}  
}
```

4.3 Vehicle Telematics Register (`data/telematics.json`)

Simulates CAN bus parameter registers from the vehicle node.

```
{  
  "v001": {  
    "online": true,  
    "battery": 78,  
    "network": "good"  
  }  
}
```

4.4 Knowledge Base (RAG) Mappings (`data/kb/`)

Files are stored under the data/kb/ directory:

- app_pairing.txt
- charging_issue.txt
- connectivity_issue.txt
- door_lock_issue.txt
- general_troubleshooting.txt
- infotainment_issue.txt
- navigation_issue.txt
- remote_control_issue.txt
- subscription_issue.txt
- vehicle_start_issue.txt

Representati

Issue Category: Door Lock Issue

Common Symptoms:

- Door not unlocking
- Door lock failure- Key fob unlock not working

Possible Causes:

1. Key fob battery dead
2. Door lock actuator failure
3. Vehicle battery low
4. Remote access module fault

Recommended Actions:

- Replace key fob battery
- Check vehicle battery
- Test manual unlocking
- Inspect actuator

5. Frontend UI Design Specification

The Streamlit UI has been upgraded to a professional dark console theme.

Key Design Enhancements:

1. Modern CSS Theme: Configures a dark slate-to-obsidian gradient background (#070913 to #0d1127) and glassmorphic telemetry grids using translucent borders (rgba(255, 255, 255, 0.06)) and blurred backdrops.
2. Typography System: Integrates Google Fonts' Outfit typeface applied across all Streamlit labels, headers, and inputs.
3. Preset Case Templates Panel: A sidebar interface displaying 4 common test presets. Clicking a preset automatically loads case parameters, triggers the graph execution, and displays results.
4. Live Vehicle Telemetry Grid: A 4-panel widget grid displaying Battery SOC, Cellular Signal, eSIM Connection status, and Sync status with dynamic state color indicators (green/yellow/red).
5. Agent Pipeline Checklist: A checklist of pipeline agents displaying status badges (COMPLETED, SKIPPED, or PENDING) and descriptions.
6. Timeline Flow Stepper: A vertical trace stepper connecting diagnostic steps in order.
7. Severity Indicator Accents: Badges and glowing borders change colors based on the classified threat levels (low = blue, medium = amber, high = orange, critical = red).
8. Interactive Database Registry Browser: A tab displaying tables of mock CRM, Subscriptions, and Telematics databases for support verification.

6. Workspace Refactoring & Cleanup Log

To clean the project repository and guarantee a solid codebase, all legacy design notes and redundant files were removed.

File Path	Original Purpose	Reason for Removal
agents/planner_agent.py	Early planner prototype.	Consolidated into `agents/intent_agent.py` (Intent+Planner LLM).
agents/rootcause_agent.py	Isolated root cause analysis.	Consolidated into `agents/investigation_agent.py` (Unified Analysis).
agents/resolution_agent.py	Isolated resolution formatting.	Consolidated into `agents/investigation_agent.py` (Unified Analysis).
test_gemini.py	LLM API token testing script.	Replaced by backend main execution.
test_qota.py	Queue/OTA test script.	Leftover development script.
test_rag.py	RAG vector search test script.	Leftover development script.
agents.md	Dev notes outlining agent categories.	Obsolete; replaced by this documentation.
agents_Work.md	Rough workspace checklists.	Obsolete developer notes.
architecture.md	System design outline notes.	Consolidated into this report.
workflow.md	Text system workflow notes.	Consolidated into this report.

The cleaned workspace directory contains only:

- agents/ (crm_agent.py, intent_agent.py, investigation_agent.py, knowledge_agent.py, subscription_agent.py, telematics_agent.py)
- data/ (mock databases & RAG manuals)
- graph/ (state.py, workflow.py)
- tools/ (mock database interfaces & RAG connectors)
- utils/ (LLM connection helper)
- app.py (Streamlit Application)
- main.py (CLI Diagnostic script)
- requirements.txt (dependencies lists)

7. Execution Guide

1. Activate Virtual Environment: `.\venv\Scripts`
- 2.Start Streamlit Service: `streamlit run`
3. Open Console Browser: `http://localhost:8501`